CitizenEase — Product & Technical Blueprint

Prepared for: Muhammad and team · 7-day hackathon MVP · Karachi, Sindh

0. Research findings you must act on first

I searched for current Domicile/PRC and CNIC requirements. **I did not find an official primary page (Sindh Home Department, a Karachi DC portal, or nadra.gov.pk) that lists a complete, current document set.** What exists is secondary: news, blogs, and low-quality guide sites that disagree with each other. Therefore:

- Every requirement in your database starts as `unverified`. Member 4 promotes a requirement to `verified` only after confirming it on an official page or an official office visit/call, and records the source.
- Nothing below is a verified government rule. All sample data is labelled **ILLUSTRATIVE**.

What the research did establish (useful for product design, with sources):

1. **Domicile and PRC are different legal instruments.** A Sindh High Court judgment states that a domicile is issued under the Pakistan Citizenship Act 1951 and Citizenship Rules 1952, while a PRC in Sindh is issued under the Sindh Permanent Residence Certificate Rules 1971 (caselaw.shc.gov.pk). This is your best answer to "what is the difference?" and a perfect first verified source. Job quotas generally need the PRC, per the same court record.
2. **Rules are in flux.** The Sindh cabinet formed a body to amend rules and curb fake domiciles/PRCs (The News, Tribune), and an inquiry blamed deputy commissioners. Expect stricter requirements. Your `last_verified` dates matter.
3. **District confusion is real.** Karachi's newly created districts (e.g. Keamari) have caused residents trouble when old-district records and new-district offices don't match (Express Tribune). District-specific handling is a genuine differentiator, but also your biggest data risk.
4. **Secondary sources disagree on details** (e.g. number of photos, CNIC address needing to match the district). Treat as leads to verify, not facts.
5. **First-time CNIC:** NADRA has changed B-Form/CNIC rules in 2025 and again in 2026 (news: ARY, ProPakistani). Reported direction: biometric B-Forms (photo and fingerprints) for children, and first-time CNIC possible via the PakID app only if

biometrics are already linked to the B-Form; otherwise visit a NADRA centre. Details such as fees and exact documents vary by source and **must be confirmed on nadra.gov.pk / the PakID portal**. Many top results are SEO content farms; do not use them as sources.

Action for Member 4 today: call/visit one Karachi DC office and one NADRA centre; photograph notice boards and forms (with permission); save URLs and dates. Official-visit evidence is a valid `source_type`.

1. Refined product specification

One-line pitch: CitizenEase tells a Karachi citizen exactly what to bring, what is still missing, and where to go, before they queue, in Urdu or English, without pretending to be the government.

What it is: a preparation and guidance layer. **What it is not:** an application submitter, a legal authority, or a data store for CNICs.

Core principles

1. Deterministic rules decide requirements. The LLM never does.
2. Every fact shows its source, status and verification date.
3. Unknown means "confirm with the office", never a guess.
4. No sensitive personal data: no CNIC numbers, scans, or names.
5. One connected journey, not isolated screens.

Primary user: a Karachi student or first-time applicant, mobile phone, may prefer Urdu.

Success metric for the demo: a judge completes questionnaire → checklist → score → AI explanation → office map in under 3 minutes with no failures.

2. Final MVP feature list

Feature	Priority	Why
A. Service discovery (cards + search + simple AI intent)	Must	Entry point
B. Guided questionnaire (data-driven, conditional)	Must	Feeds rules

Feature	Priority	Why
C. Personalized checklist + rules engine	Must	Differentiator
D. Readiness score	Must	Cheap once C exists
E. AI assistant (RAG over verified records)	Must (scoped)	Required "AI integration"
F. Office locator (curated, map + fallback list)	Must	Completes journey
Urdu + English interface	Must	Core accessibility; UI strings only, not full CMS translation
Checklist tick-off progress tracker	Must	It is the readiness input
Source/verification page	Must	Trust
Checklist PDF download	Should	Use browser print stylesheet, not a PDF library
Document explanations	Should	Free: it is the `why_needed` field
Saved progress	Should	Anonymous session ID + localStorage
Voice input	Later	Browser Web Speech API is unreliable for Urdu; demo risk
Regional languages (Sindhi, etc.)	Later	Requires verified translation
Document OCR/scanning	Later (do not build)	Sensitive data, high complexity
Login / accounts	Later	Not needed; avoid PII

3. Feature-by-feature implementation guide

A. Service discovery. Two services in a `services` table rendered as cards. Search is client-side filter over title + keywords. AI discovery: one endpoint takes free text and returns `{service_slug, confidence, clarifying_question}`. Constrain the model to

choose from the list of service slugs (enum) or `unknown`. For "I need proof I belong to Karachi", return `domicile_prc` with a clarifying question about purpose (job/admission/other). Never decide legal eligibility.

B. Questionnaire. Questions are rows, not code. Each question has a key, type (`single_choice`, `boolean`, `district`), and options. Conditions say when it is shown. The frontend renders one question at a time from the API. Back button re-posts earlier answers; the server recomputes visibility. Validate on both sides (Pydantic on the server).

C. Checklist. Rules evaluate answers → list of applicable requirements, each with status (`mandatory`, `conditional`, `optional`) and verification (`verified`, `unverified`). The user marks each as Have / Don't have / Not sure.

D. Readiness score. Only `verified` + `mandatory` + applicable items count.

- Score = ready ÷ total, × 100.
- If total is 0, return `score = null` and display "No verified requirements yet. Confirm with the office" (never 0% or 100%).
- Show unverified items in a separate block, not in the denominator.
- Always show the disclaimer: this is preparation, not approval.

E. Assistant. Detailed in section 9.

F. Offices. Hand-curated table, Leaflet map, list view as fallback. Detailed below.

4. System architecture

Pattern: modular monolith. One FastAPI app, one database, one deploy.

```
Browser (Next.js, mobile-first)
   |  HTTPS JSON
   ▼
FastAPI app
 ├─ routers/        (HTTP only: parse, validate, call service)
 ├─ modules/
 |   ├─ catalog      services, sources, fees, offices (read-only)
 |   ├─ questionnaire question flow + answer storage
 |   ├─ rules        pure functions, NO database/AI imports
 |   ├─ checklist    builds checklist from rules output
 |   ├─ readiness    score calculation (pure)
 |   └─ assistant    retrieval + prompt + LLM client + guardrails
 └─ db/             SQLAlchemy/psycopg or supabase-py access
   ▼
Supabase PostgreSQL
```

Boundary rules (prevent chaos in a team of four)

- `rules` and `readiness` are pure Python: input dicts → output dicts. Testable with no network.
- `assistant` may *read* checklist output but can never write requirements.
- Routers contain no business logic.
- Frontend talks only to FastAPI. It never talks to Supabase or the LLM directly. This keeps keys and logic server-side.

Data flow (happy path): frontend `POST /questionnaire/start` → server returns session + first question → `POST /questionnaire/answer` repeatedly → `POST /checklist/generate` (server loads answers + requirements → rules → checklist rows → readiness) → frontend renders → `POST /assistant/chat` with `checklist_id` → server retrieves verified records → LLM explains.

Authentication: none for the MVP. Use an anonymous `session_id` (random UUID, held in localStorage and passed as a header). Rate-limit by IP + session. No PII means no auth burden. If you later add login, use Supabase Auth.

Security: secrets only in backend env vars; CORS restricted to your Vercel domain; Pydantic validation with length limits; RLS on Supabase with the backend using the service key (never shipped to the browser); do not log full chat contents; strip anything that looks like a CNIC number (13 digits) from chat input before sending to the LLM and show a notice "Don't enter your CNIC number".

Error handling: a uniform error body `{error: {code, message, retryable}}`; frontend shows a friendly state and a retry; the assistant failing must never block the checklist.

5. Technology stack: evaluation

Choice	Verdict	Reason
Next.js + React + TS + Tailwind + shadcn/ui	Keep, with a caution	You have basic React. Use the App Router minimally (client components, `fetch` to FastAPI). Skip server actions, SSR data tricks, and auth libraries. TypeScript: keep, but allow `any` sparingly rather than lose a day.
FastAPI + Pydantic	Keep	Fast to write, automatic OpenAPI docs, which let frontend generate/mocks types.
Supabase Postgres	Keep	Free tier, GUI table editor (Member 4 can edit data without SQL), easy seed. Use it as plain Postgres.
Groq (primary) + OpenRouter (fallback)	Keep	Groq is fast; OpenRouter gives alternate models. Put both behind one `LLMClient` interface with automatic failover. Check current free-tier limits and model names on their sites, they change often.
Leaflet + OpenStreetMap	Keep	No API key. Use only tiles within OSM usage policy (fine for a demo).
OSRM	Change	The public OSRM demo server is not for production and can be slow or down. **Recommended:** skip routing; use "Open in Google Maps / OpenStreetMap directions" deep links built from office coordinates. Zero dependency, always works. Optionally try OSRM and fall back to the link.

Choice	Verdict	Reason
Vercel + Render	Keep, with caution	Render free tier sleeps and cold-starts (~30–60 s). Mitigate: ping `/health` every 10 min and warm the server before the demo.

Avoid: vector database, LangChain/LlamaIndex, Redis, Celery, Docker orchestration, microservices, auth, file uploads, GraphQL, custom ML.

Environment variables (backend): `DATABASE_URL`, `SUPABASE_SERVICE_KEY`, `GROQ_API_KEY`, `OPENROUTER_API_KEY`, `ALLOWED_ORIGINS`, `ENV`. **Frontend:** `NEXT_PUBLIC_API_BASE_URL` only. Commit `.env.example`, never `.env`. Store production values in Vercel/Render dashboards. Rotate any key that touches a screenshot or chat.

6. Database schema (PostgreSQL)

Conventions: `id uuid primary key default gen_random_uuid()`, `created_at timestamptz default now()`, text slugs for stable references.

Core catalog

Table	Purpose	Key columns
`services`	The two services	`slug` unique (`domicile_prc`, `first_cnic`), `title_en`, `title_ur`, `description_en`, `description_ur`, `keywords text[]`, `is_active`
`application_types`	Variants within a service (e.g. Domicile only / PRC only / both)	`service_id` FK, `slug`, `title_en`, `title_ur`

Table	Purpose	Key columns
`sources`	Where a fact comes from	`title`, `url`, `issuing_authority`, `source_type` (`official_web`, `official_document`, `office_visit`, `court_ruling`, `news_secondary`), `verification_status` (`verified`, `unverified`, `outdated`, `conflicting`), `last_verified_on date`, `verified_by`, `applicable_district` (nullable = all), `service_id` FK, `notes`, `limitations`
`requirements`	One document/step	`service_id` FK, `application_type_id` FK nullable, `code` unique, `title_en`, `title_ur`, `why_needed_en`, `why_needed_ur`, `kind` (`document`, `step`), `verification_status`, `source_id` FK, `last_verified_on`, `notes`
`eligibility_rules`	When a requirement applies	`requirement_id` FK, `applicability` (`mandatory`, `conditional`, `optional`), `condition jsonb`, `district` nullable, `priority int`, `is_active`
`fees`	Fees (only if verified)	`service_id` FK, `application_type_id` nullable, `amount_pkr numeric nullable`, `label`, `source_id` FK, `verification_status`. A null amount means unknown; UI shows "Confirm with office".

Table	Purpose	Key columns
`offices`	Curated offices	`name`, `office_type`, `district`, `address`, `lat`, `lng`, `services text[]`, `source_id` FK, `verification_status`, `last_verified_on`, `hours_note nullable`, `notes`. No `is_operating` column on purpose.

Questionnaire

Table	Purpose	Key columns
`questions`	One question	`service_id` FK, `key` (e.g. `district`), `text_en`, `text_ur`, `input_type`, `order_index`, `required bool`, `validation jsonb`
`question_options`	Choices	`question_id` FK, `value`, `label_en`, `label_ur`, `order_index`
`question_conditions`	Show-if logic	`question_id` FK, `depends_on_key`, `operator` (`eq`, `neq`, `in`), `value jsonb`

Runtime (user data, no PII)

Table	Purpose	Key columns
`user_sessions`	Anonymous session	`id`, `service_id`, `language`, `expires_at`
`questionnaire_answers`	Answers	`session_id` FK, `question_key`, `answer jsonb`, unique (`session_id`, `question_key`)
`generated_checklists`	One generated result	`id`, `session_id` FK, `service_id`, `rules_version text`, `answers_snapshot jsonb`, `readiness jsonb`

Table	Purpose	Key columns
`checklist_items`	Items	`checklist_id` FK, `requirement_id` FK, `applicability`, `verification_status`, `user_status` (`have`, `missing`, `not_sure`), `explanation_snapshot`

Relationships: services 1—N application_types, requirements, questions, fees, sources; requirements 1—N eligibility_rules; requirements N—1 sources; sessions 1—N answers; sessions 1—N checklists; checklists 1—N items.

Condition format (one jsonb shape everywhere):

```
{"all": [ {"key": "first_application", "op": "eq", "value": true},
          {"key": "district", "op": "in", "value":
["karachi_east","karachi_south"]} ]}
```

Support only `all`, `any`, `eq`, `neq`, `in`. Nothing more.

ILLUSTRATIVE sample records (not real rules):

```
sources: {title:"SAMPLE – Sindh PRC Rules 1971 (court summary)",
source_type:"court_ruling", verification_status:"unverified"}
requirements: {code:"SAMPLE_CNIC_COPY", title_en:"Copy of applicant CNIC
(ILLUSTRATIVE)", verification_status:"unverified"}
eligibility_rules: {applicability:"mandatory", condition:{"all":
[{"key":"has_cnic","op":"eq","value":true}]}}
```

Outdated and conflicting sources

- A source past a threshold (suggest 90 days) is auto-flagged `stale` by a query; the UI shows "Last verified: date. May have changed."
- Conflicts: set both sources `conflicting`, mark the requirement `unverified`, show the "confirm with office" banner. Never silently choose a winner. The *more authoritative and more recent official* source wins only after a human (Member 4) resolves it and records the note.
- Keep history: never overwrite; add a new source row and re-point the requirement.

7. API design

Base: `/api/v1`. All responses JSON. Errors: `{"error":{"code":"...","message":"...","retryable":false}}`. Standard codes: `400 validation_error`, `404 not_found`, `409 conflict`, `422 schema_error`, `429 rate_limited`, `502 provider_unavailable`, `500 internal`.

Endpoint	Purpose	Request	Response	Validation / errors
`GET /services`	List services (+ `?q=` search)	query `q`, `lang`	`[{id, slug, title, description, icon}]`	`lang` in `en`, `ur`
`GET /services/{slug}`	Detail + application types + source summary	path slug	`{service, application_types, source_summary: {verified_count, unverified_count, last_verified_on}}`	404 unknown slug. **Use slug, not id.**
`POST /questionnaire/start`	Create session, return first question	`{service_slug, language}`	`{session_id, question, progress}`	404 service
`POST /questionnaire/answer`	Save answer, return next question or `done`	`{session_id, question_key, answer}`	`{next_question | done:true, progress, visible_keys}`	answer must match question type/options; 400 if invalid; supports re-answering (upsert) which recomputes downstream visibility and drops now-hidden answers
`POST /checklist/generate`	Run rules, create checklist	`{session_id}`	`{checklist_id, items[], readiness, disclaimers[]}`	409 if questionnaire incomplete
`GET /checklist/{id}`	Fetch saved checklist	path	same as above	404
`PATCH /checklist/{id}/items`	Update have/missing, return new score	`{updates: [{item_id, user_status}]}`	`{items, readiness}`	status enum check; batch in one call

Endpoint	Purpose	Request	Response	Validation / errors
`POST /assistant/chat`	Ask the assistant	`{session_id, checklist_id?, message, language}`	`{answer, citations[{source_id,title,url,last_verified_on}], needs_confirmation, suggested_actions[]}`	message ≤ 500 chars; 429 on limits; 502 → returns fallback payload (see §9)
`GET /offices`	List offices	`service, district`	`[{id,name,address,lat,lng,district,verification_status,last_verified_on,notes}]`	
`GET /fees/{service_slug}`	Fees	path	`[{label, amount_pkr | null, verification_status, source}]`	null → "confirm"
`GET /sources/{service_slug}`	Sources	path	list with status, dates	
`GET /health`	Warm-up/uptime	none	`{ok:true}`	
`POST /discover` (**added**)	AI intent → service	`{text, language}`	`{service_slug | null, confidence, clarifying_question}`	

Recommended changes: use slugs instead of ids in public URLs; drop nothing, but merge fees and sources into the service detail response if time is short (keep endpoints, but the frontend can use one call); add `/health` and `/discover`.

Checklist item response shape:

```
{"item_id":"...","title":"...","why_needed":"...","applicability":"mandatory"
,
 "verification_status":"unverified","user_status":"missing",
 "source":{"title":"...","url":"...","last_verified_on":"2026-11-08"}}
```

Readiness response shape:

```
{"score":67,"ready":2,"total":3,"missing":["..."],"conditional":["..."],
 "unverified":["..."],"explanation":"2 of 3 verified required items
ready.","is_guarantee":false}
```

8. Rules engine design

Design: data-driven rows + one small pure evaluator. Options compared:

Option	Pros	Cons	Verdict
Hardcoded `if` statements in Python	Fastest to start	Every rule change = code deploy; non-developers can't edit; hard to show sources	Reject
Rules as DB rows evaluated by a tiny interpreter	Member 4 edits data; each rule carries its source; testable	Slightly more upfront work	**Recommend**
Third-party rules engine / DSL	Powerful	Learning curve, overkill	Reject

Algorithm (about 60 lines of Python):

1. Load active rules + requirements for the service (and application type).
2. For each rule, evaluate its `condition` against the answers dict. Missing answer key → condition is **unknown**.
3. If the rule's `district` is set, it applies only when the district matches.
4. Output per requirement: `applies` (true/false/unknown), `applicability`, `verification_status`.
5. Resolve duplicates/conflicts: if two rules give different applicability for one requirement, take the more specific (district-specific over general); if still tied, mark `unverified`.
6. `unknown` → place in a "Confirm with office" group; never silently drop it.
7. Return a `rules_version` (hash of rule rows) stored with the checklist for reproducibility.

Keep the evaluator free of database and AI imports so it is unit-testable with plain dictionaries.

9. AI integration and safety design

Workflow: intent text → `/discover` (LLM picks a service slug from an enum) → questionnaire (no LLM) → rules → checklist → readiness → assistant explains using retrieved records → follow-ups.

Retrieval (RAG without a vector DB). You have two services and maybe 30–50 records. Do not build embeddings. Instead:

1. Retrieve by **structure**: for a chat turn, load (a) the user's checklist items with their source rows, (b) fees, offices and sources for that service, (c) a small curated FAQ table (e.g. Domicile vs PRC) written by Member 4 with sources.
2. If the question mentions a specific item or topic, filter by keyword/Postgres full-text search over `requirements`, `faq`, `sources`.
3. Pass the top records (cap ~3,000 tokens) to the model as a numbered **CONTEXT** block with ids.
4. Upgrade to pgvector later only if the FAQ grows large.

Prompt structure (every call): system prompt (fixed rules) → CONTEXT block (records with ids, status, date) → applicant summary (answers as key/values, no PII) → last 6 messages → user message. Keep temperature low (0.1–0.2).

Structured output. Ask for JSON only:

```
{"answer":"...", "used_record_ids":["R3","S1"],
"needs_official_confirmation":true, "language":"ur"}
```

Use the provider's JSON mode if available, then validate with Pydantic. On parse failure: retry once with "return valid JSON only"; if it fails again, return a safe fallback answer built from templates. Citations shown in the UI are **built by the server from `used_record_ids`**, not typed by the model, so the model cannot fabricate a source.

Hallucination controls (layered):

1. Answer only from CONTEXT; the prompt says so explicitly.
2. Server-side check: any `used_record_id` not in the context → discard the answer.
3. Output filter: if the answer contains a currency amount (Rs/PKR digits), a time-of-day, or a phone number that does not appear verbatim in CONTEXT → replace with the "confirm with office" fallback. This catches invented fees/timings cheaply.
4. Anything about legal eligibility → refuse and say to confirm with the office.
5. Rules engine output is passed as ground truth; the model is told it cannot add or remove requirements.
6. Automated red-team tests (section 13) run before every deploy.

Missing information: the model must say "This is not in the verified information I have. Please confirm with [office/NADRA]." and set `needs_official_confirmation: true`. The UI then shows a banner.

Conversation context: stateless server; client sends `session_id`; server loads the checklist and the last 6 messages (stored in memory or a small `chat_messages` table, optional). Do not feed old chat as authoritative facts.

Provider failures and rate limits: `LLMClient` tries Groq → OpenRouter → static fallback. Timeout 12 s, one retry, exponential backoff on 429. Static fallback: show the checklist item's stored `why_needed` text and the sources panel ("AI is temporarily unavailable; here is the verified information"). A per-session cap (e.g. 20 messages) protects free-tier quotas. Cache answers to identical (service, question-key) FAQ questions.

Sample system prompt (adapt wording; test in both languages):

```
You are CitizenEase Assistant, a guide that helps people in Pakistan prepare
for
government services. You are NOT a government employee and NOT a lawyer.

RULES (never break these):
1. Use ONLY the facts in CONTEXT. Each record has an id, a verification
status,
   and a last-verified date.
2. Never invent requirements, fees, office timings, phone numbers, addresses,
   legal rules, or processing times. If it is not in CONTEXT, say it is not
   available and that the user must confirm with the relevant office or the
   official website.
3. Records with status "unverified" must be described as "not yet verified".
   Never call them official.
4. The checklist comes from a rules engine. Do not add, remove, or change
   requirements. You may only explain them.
5. Do not decide whether the user is legally eligible. Say that the issuing
   authority decides.
6. Never ask for CNIC numbers, phone numbers, or documents. If the user
shares
   them, tell them not to.
7. Reply in the user's language (English or Urdu). Keep answers short, in
plain
   words, with at most 5 steps.
8. Always list the ids of the records you used.

OUTPUT: valid JSON only:
```

```
{"answer": str, "used_record_ids": [str], "needs_official_confirmation":
bool}

CONTEXT:
{{records}}
APPLICANT SUMMARY:
{{answers}}
```

Urdu note: generate Urdu answers from verified English records; do **not** store machine-translated requirement text as verified. Have a native speaker on the team review Urdu UI strings and `why_needed_ur`.

10. UI/UX screen plan

Design system: deep forest green (primary), warm off-white (background), muted teal (accent), amber for "unverified/confirm", red only for errors. Inter for English; Noto Nastaliq Urdu for Urdu (use larger line-height, about 2.0, since Nastaliq needs vertical room). Mobile-first (design at 375 px). RTL support via `dir="rtl"` and Tailwind logical utilities (`ms-`, `me-`, `ps-`). Minimum 16 px text, 44 px touch targets, WCAG AA contrast. **Never use colour alone** to convey status: add an icon and a label (Verified / Unverified).

Global components: language toggle (EN/اردو) in the header; persistent footer disclaimer "Guidance only. Not an official government service."; trust badge on every fact (status + date).

Screen	Main components	User actions	Data	Navigation	Loading / error
1 Landing	Hero, one-line value, search/ask box, two service cards, "how it works", disclaimer	Type a need, pick service	`GET /services`	→ Discovery or Questionnaire	Skeleton cards; static fallback cards if API down
2 Discovery	Search, cards, AI result with clarifying question	Search, describe problem, choose	`/services`, `/discover`	→ Service detail	Retry button; AI failure falls back to manual cards

Screen	Main components	User actions	Data	Navigation	Loading / error
3 Service detail	What it is, who typically needs it, Domicile vs PRC explainer, verification summary, "Start" button	Start questionnaire	`/services/{slug}`	→ Questionnaire	Error card + retry
4 Questionnaire	Progress bar, one question per screen, Back/Next, inline validation, review step	Answer, go back, edit	`/questionnaire/*`	→ Checklist	Disable Next while saving; keep local copy of answers if request fails
5 Checklist	Grouped sections (Required, Conditional, Optional, Confirm with office), item cards with Have / Missing / Not sure, "why" expander, source chip	Toggle status, open source, ask AI about item	`/checklist/*`	→ Score (same page header), Assistant, Offices	Optimistic updates; revert on error
6 Readiness	Ring/percentage, counts, "how calculated", missing list, unverified list, disclaimer	Jump to missing items	from checklist response	Part of checklist page (not a separate load)	`null` score state "No verified requirements yet"
7 Assistant	Chat panel, suggested questions, citation chips, confirmation banner, language toggle	Ask, tap citations	`/assistant/chat`	Slide-over from checklist so context persists	Typing indicator; provider-down banner + static explanation

Screen	Main components	User actions	Data	Navigation	Loading / error
8 Office locator	Filters (service, district), map, list, office card (address, verification, directions link)	Filter, select marker, open directions	`/offices`	From checklist "Where to submit"	If map tiles fail, show list with address and directions links; no offices → "We couldn't find a verified office; check the official site"
9 Sources	Table of sources: title, authority, status, last verified, notes	Open source	`/sources/{slug}`	Linked from every chip	Empty state explains nothing is verified yet
10 Saved progress (optional)	Resume checklist by session	Resume	`GET /checklist/{id}` from localStorage	Landing "Continue" button	Expired → start over

Connected flow: Landing → Service → Questionnaire → Checklist (+score) → Assistant / Offices, with a persistent step indicator. Build the checklist page as the hub; the assistant and the map open from it.

11. Monorepo folder structure

```
citizenease/
├─ README.md  · .gitignore  · .env.example
├─ docs/               (sources log, API contract, decisions)
├─ data/
|   ├─ seed/          (services.json, questions.json, requirements.json,
|   |                  rules.json, sources.json, offices.json, faq.json)
|   └─ test_cases/    (checklist_cases.yaml, assistant_redteam.yaml)
├─ frontend/          (Next.js)
|   ├─ src/app/       (routes: /, /services, /services/[slug],
|   |                  /questionnaire, /checklist/[id], /offices, /sources)
|   ├─ src/components/ (ui/, checklist/, questionnaire/, assistant/, map/)
|   ├─ src/lib/       (api.ts, types.ts, i18n/{en,ur}.json, mock/)
|   └─ public/fonts/
├─ backend/
|   ├─ app/
```

```
|   |   ├─ main.py
|   |   ├─ config.py
|   |   ├─ routers/    (services.py, questionnaire.py, checklist.py,
|   |   |               assistant.py, offices.py, fees.py, sources.py)
|   |   ├─ modules/    (catalog/, questionnaire/, rules/, checklist/,
|   |   |               readiness/, assistant/)
|   |   ├─ schemas/    (Pydantic models = API contract)
|   |   └─ db/         (client.py, queries.py, migrations/)
|   ├─ scripts/        (seed.py, check_sources_stale.py)
|   └─ tests/          (test_rules.py, test_readiness.py, test_api.py,
|                       test_assistant_guardrails.py)
```

Seed data in JSON files committed to git is your source of truth; `seed.py` loads it into Supabase. This gives you version history of every requirement edit and lets the team work without a shared database.

12. Seven-day roadmap

Assume two to three working sessions per day; adjust.

Day 1 — Research, contracts, setup

- Tasks: official-source research (M4); create repo, branches, CI-free setup; Supabase project; FastAPI skeleton with `/health`; Next.js skeleton; freeze API schemas (Pydantic = contract) and publish mock JSON; decide the 2 services' question lists.
- Deliverables: `sources.json` first draft, API contract doc, running skeletons, deployed hello-world on Vercel and Render.
- Dependencies: none. Acceptance: both skeletons deploy; mock JSON exists for every endpoint.
- Risks: no official sources found. Mitigation: use office-visit evidence; mark unverified.
- Must work by end: deploy pipeline and shared contract.

Day 2 — Database and rules engine

- Tasks: schema migrations; seed script; rules evaluator and readiness calculator with unit tests; first requirement records (unverified) for both services.
- Acceptance: `pytest` passes for the test cases (first-time domicile, different district, no verified requirements); seed loads from scratch in one command.
- Risks: rule format churn. Mitigation: the condition format is frozen (section 6).
- Must work: `POST /checklist/generate` with hardcoded answers returns a checklist.

Day 3 — Frontend and questionnaire

- Tasks: questionnaire API endpoints; question renderer, progress bar, back navigation, validation; checklist page against mocks, then real API; EN/UR strings skeleton.
- Acceptance: complete questionnaire → real checklist on a phone-sized screen.
- Risks: conditional logic bugs. Mitigation: test with the rules test cases.
- Must work: end-to-end journey without AI.

Day 4 — AI assistant

- Tasks: `LLMClient` with Groq + OpenRouter fallback; retrieval; prompt; JSON validation; output filters; chat UI with citation chips; `/discover`.
- Acceptance: red-team cases pass (invented requirement refused, unknown fee → confirm); provider-down shows fallback.
- Risks: rate limits, JSON failures. Mitigation: low temperature, retry, static fallback.
- Must work: grounded answers with server-built citations.

Day 5 — Offices, fees, sources

- Tasks: curated `offices.json` (verified by M4 with evidence), Leaflet map, list fallback, directions deep links; fees (null if unverified); sources page.
- Acceptance: map works; unplugging the map still shows the list; every displayed fact shows status/date.
- Risks: office data unverified. Mitigation: show an "unverified" label; fewer offices, correct beats many.
- Must work: connected journey to the office.

Day 6 — Testing, integration, deployment

- Tasks: run the full acceptance suite (section 13); fix bugs only; Urdu review; mobile pass; production env vars; warm-up ping; print stylesheet for checklist PDF.
- Acceptance: all 12 scenarios pass on the deployed site.
- Risks: integration bugs. Mitigation: no new features after noon.
- Must work: production URL stable.

Day 7 — Freeze, polish, demo

- Tasks: demo data check; rehearsal x3; slides; backup screen recording; offline fallback (screenshots); README; final source verification pass.
- Acceptance: a team member who did not build the feature can run the demo.
- Rule: **code freeze at the start of the day**; only P0 bugs.

Cut list (in this order if behind): 1) checklist PDF, 2) saved progress page, 3) AI `/discover` (use search only), 4) Urdu for assistant answers (keep UI Urdu), 5) map (keep list + directions links), 6) second service's depth (keep CNIC as a smaller, clearly labelled flow), 7) fees page. **Never cut:** rules engine, checklist, readiness, source labels, AI guardrails.

13. Team structure

Member	Owns	Primary folders	Hands off to
1 Frontend	Screens, i18n, map UI, mobile polish	`frontend/`	Consumes API contract
2 Backend + DB	Routers, schema, seed, deployment, questionnaire API	`backend/app/routers`, `db`, `data/seed`	Publishes endpoints and OpenAPI
3 AI + rules	`modules/rules`, `readiness`, `assistant`, tests	`backend/app/modules`, `tests`	Pure-function interfaces
4 Research + QA + docs	Sources, requirements data, offices data, test cases, demo script, README	`data/`, `docs/`	Data to M2 via seed JSON

Shared interfaces (freeze on Day 1): Pydantic schemas in `backend/app/schemas/`; seed JSON formats; the condition format; the error format; the checklist/readiness response shapes.

Parallel development with mocks: the frontend reads `NEXT_PUBLIC_USE_MOCKS=true` and `src/lib/mock/*.json` that exactly match the schemas; flip the flag when the real endpoint is ready. Backend modules are tested with plain dictionaries before routes exist.

Daily integration: a 15-minute sync at start and a mandatory merge to `main` before the end of the day; each person demonstrates their slice working on the deployed environment.

Avoiding merge conflicts: one owner per folder; feature branches `feat/<name>-<topic>`; small PRs (review by another member, one-line approval); never commit directly to `main`; seed JSON edits by M4 only (others request changes); rebase before merging; lockfiles

changed only by one person per day.

Team of three: merge Member 4's engineering tasks into the others and keep a **research owner**: Member 2 takes research/data (they own the seed data anyway), Member 3 keeps AI + rules + QA, Member 1 keeps frontend + demo script. Cut the checklist PDF, saved progress, and Urdu assistant answers up front; keep UI Urdu only if a native speaker is on the team.

14. Testing and reliability

Test the rules engine independently: unit tests call `evaluate(answers, rules)` with dictionaries; no network, no LLM, no database. A YAML file lists `answers → expected requirement codes → expected readiness`. Run on every commit.

Simple test dataset: 12–15 answer profiles (first-time, repeat, student, each Karachi district, married applicant, no CNIC / B-Form only, missing answer keys) plus 20 assistant prompts (10 normal, 10 adversarial).

#	Scenario	Pass criteria
1	First-time Domicile/PRC applicant	Checklist matches the seeded rules; each item shows source status
2	Different Karachi district	District-specific rule applied; general rule otherwise
3	First-time CNIC applicant	Flow shows the "confirm with NADRA" limitation banner
4	Missing documents	Score < 100; missing list correct
5	Unverified requirement	Appears in a separate section; not counted in the score
6	No verified mandatory requirements	`score = null`, explanatory message, no 0% or 100%
7	AI asked to invent a requirement	Refusal + confirmation notice; no new item
8	AI asked about an unknown fee	"Fee not verified; confirm with office"; no number
9	Office unavailable	Message with official-site pointer

#	Scenario	Pass criteria
10	Map API unavailable	List and directions links still work
11	AI provider unavailable	Static explanation + sources; checklist unaffected
12	Mobile responsiveness	375 px usable; no horizontal scroll; Urdu RTL correct

Measurable quality metrics: rules tests 100% pass; hallucination red-team: 0 invented fees/requirements out of 10 adversarial prompts; every assistant answer with facts has ≥ 1 citation; 100% of checklist items show status and date; p95 checklist generation < 1 s; assistant response < 6 s typical; questionnaire completion by a new tester in < 2 min; 3 non-team users complete the journey without help.

15. Risk register

Risk	Prob.	Impact	Prevention	Fallback
Outdated government information	High	High	`last_verified_on` on every record, stale flag after 90 days, visible dates	Show "may have changed; confirm" banner
District-specific requirements unknown (also new Karachi districts)	High	High	Start with one or two districts verified well; mark others unverified	Generic checklist + "confirm with your district DC office"
Incorrect AI answers	Medium	High	RAG over verified records, JSON output, server-built citations, output filters, red-team tests	Static explanation from stored text
Unverified office details	Medium	Medium	Curated list with evidence; no "currently operating" claims	Hide unverified office or label it

Risk	Prob.	Impact	Prevention	Fallback
Provider downtime / rate limits	Medium	Medium	Two providers, retries, caching, per-session cap	Static fallback
Sensitive personal data	Low	Very high	No PII fields, no uploads, strip 13-digit numbers, minimal logging	Delete session data on request
Integration delays	High	High	Day-1 contracts, mocks, daily merge	Cut list
Overengineering	High	Medium	Modular monolith, no vector DB, no auth	Reduce scope per cut list
Render cold start in demo	High	Medium	Warm-up ping; pre-open pages	Run locally on a laptop with the seed DB
Legal-information liability	Low	High	Persistent disclaimer; no eligibility decisions; no claims of approval	Remove claims, keep sources
Conflicting or low-quality sources (SEO farms)	High	High	Source whitelist: government sites, court records, official visits	Mark `conflicting`, defer to office
Urdu quality/RTL bugs	Medium	Medium	Native review; limited Urdu surface	English-only assistant

16. Hackathon demo flow

Setup: use the seeded demo dataset. Be honest on screen: if requirements are not verified, the app shows them as **unverified** and your pitch says so. That honesty *is* a feature judges can see. Where Member 4 has verified items from official evidence, show a few as "Verified" with date and source to demonstrate the full model.

Script (about 4 minutes):

1. *Problem (20 s):* wasted visits and incomplete files.

2. *Discovery (30 s):* type "I need proof that I belong to Karachi" → AI suggests Domicile/PRC and asks about the purpose; show the Domicile vs PRC explanation grounded in the court-ruling source.
3. *Questionnaire (40 s):* student, first application, district chosen; show a conditional question appearing and going back to change an answer.
4. *Checklist (40 s):* grouped items, mark two as "Have", others "Missing"; show the "unverified" section and source chips.
5. *Readiness (20 s):* the score and "how calculated"; point out it is not an approval guarantee.
6. *Assistant (50 s):* ask in Urdu "Why do I need this document?" (citation shown), then "What is the fee?" → "not verified; confirm" (show the guardrail).
7. *Office (30 s):* open the office, map, directions link; mention the list fallback.
8. *Close (20 s):* architecture in one slide: deterministic rules + AI explainer; what is next.

Safeguards: backup video, localhost copy, pre-warmed servers, pre-created session, screenshots for each screen.

17. Recommended development order and feature cut list

Build order (dependency chain): contracts and schema → seed data and sources → rules + readiness (tests) → questionnaire API → checklist API → frontend questionnaire + checklist → assistant (retrieval, guardrails) → offices and map → sources page → i18n polish → PDF/print → saved progress.

Dependencies at a glance: assistant needs checklist + sources; readiness needs rules; offices are independent (can start Day 3 in parallel); i18n touches everything, so use string keys from day one.

First 3 things to do today: (1) Member 4 starts official verification calls/visits and logs sources; (2) Members 2 and 3 agree the Pydantic schemas and condition format; (3) Member 1 sets up the Next.js shell with EN/UR and mock data.

Final honest caution: the product's credibility depends on the data, not the AI. A smaller set of well-verified requirements and a clearly labelled unverified remainder will beat a long, confident checklist that turns out to be wrong.